

Queens Problem

Approccio quantistico: oracle design per l'algoritmo di Grover

1. Introduzione

Il queens problem è un classico esempio di problema di soddisfacimento di vincoli (CSP); per tale ragione, si presta molto bene ad essere implementato come oracolo per l'algoritmo di Grover, che è in grado di riconoscere le soluzioni che soddisfano vincoli complessi, anche in spazi di ricerca molto grandi (a meno di limiti fisici dei computer quantistici).

2. Scelte progettuali

Per la realizzazione della logica dell'oracolo, si è adottato in una prima fase un approccio bottom-up, ovvero si è implementato e visualizzato direttamente in Qiskit il caso più semplice (4x4) per poi astrarne delle regole per il caso generale (NxN), come quelle per il dimensionamento del circuito e per la costruzione dei suoi componenti.

Successivamente, si è utilizzato l'ambiente Prolog, basato sul framework Quantum, per l'implementazione del caso generale, in modo da passare a un approccio definitivo top-down; Prolog restituisce il circuito dell'oracolo, Qiskit lo esegue utilizzando l'algoritmo di Grover.

La progettazione segue i seguenti punti:

- le righe delle regine sono fissate a-priori, ovvero l'indice di riga corrisponde a quello della regina stessa;
- lo stato è rappresentato dai valori di colonna delle regine, opportunamente codificati a seconda della dimensione della scacchiera;
- il problema è stato scomposto in tre componenti per necessità computazionali, ovvero per un compromesso tra ampiezza e profondità circuitale, ottenuto attraverso il riutilizzo di ancilla qbits, che vengono "puliti" nelle fasi di uncompute;
- i tre componenti sono:
 - o vincolo sulle colonne
→ $C_i \neq C_j, i, j \in [0, N-1], i \neq j$
 - o vincolo sulle diagonali differenza (o principali)
→ $R_i - C_i \neq R_j - C_j, i, j \in [0, N-1], i \neq j$
 - o vincolo sulle diagonali somma (o secondarie)
→ $R_i + C_i \neq R_j + C_j, i, j \in [0, N-1], i \neq j$
- ogni componente effettua tanti controlli di "collisione" (ovvero i casi di uguaglianza, per i quali il vincolo non è rispettato) quante sono le coppie possibili di regine e infine effettua un test di soddisfacimento del vincolo controllando che non ci sia stata alcuna collisione;
- i controlli di collisione avvengono confrontando in ordine ciascuna regina, eccetto l'ultima, con tutte le successive ($N*(N-1)/2$ confronti);
- la singola collisione tra due regine viene verificata con l'implementazione di un operatore di uguaglianza (EQ), ovvero di uno XOR bitwise seguito da un NOR dei qbits risultanti;

- per il vincolo sulle diagonali differenza/somma si è utilizzata l'accumulazione di un offset positivo/negativo di colonna (con controllo di overflow), con "reset" al termine dei controlli di collisione per una data regina;
- il test parziale, ovvero relativo a uno dei vincoli, viene effettuato con l'implementazione di un operatore NOR sui qbits che memorizzano la presenza di collisioni;
- i test parziali relativi ai tre componenti vengono infine messi in AND nel test globale per determinare se tutti i vincoli sono rispettati e di conseguenza lo stato rappresenta una soluzione del problema;
- uncompute globale dei tre componenti.

In seguito, sarà illustrata la progettazione dell'oracolo secondo le seguenti fasi:

- Inizializzazione
- Vincolo sulle colonne
- Vincolo sulle diagonali
- Test di soddisfacimento dei vincoli
- Composizione dell'oracolo

3. Inizializzazione

3.1. Dimensionamento del circuito

Data una scacchiera $N \times N$, il dimensionamento del circuito include le seguenti clausole:

- **queenDIM**: ritorno del numero di qbits necessari alla rappresentazione dello stato di una singola regina (ovvero alla codifica dei valori delle colonne), nonché, per costruzione, alla memorizzazione degli xor bitwise tra una coppia di regine ($\text{QueenDIM} = \lceil \log_2(N) \rceil$);
- **stateDIM**: ritorno del numero di qbits necessari alla rappresentazione dello stato di tutte le regine, spazio di ricerca per l'algoritmo di Grover ($\text{StateDIM} = N * \text{QueenDIM}$);
- **checksDIM**: ritorno del numero di ancilla qbits necessari al confronto di tutte le possibili coppie di regine ($\text{ChecksDIM} = N * (N-1) / 2$);
- **auxDIM**: ritorno del numero di ancilla qbits necessari alle operazioni intermedie delle catene Toffoli, con calcolo basato su $C_{\max}$, numero massimo di controlli presenti nelle n-tof del circuito ($\text{AuxDIM} = C_{\max} - 2$, con $C_{\max} = \text{ChecksDIM}$);
- **dim**: ritorno del numero totale di qbits necessari al circuito, dato dalla somma dei precedenti qbits e da qbits indipendenti da N , ovvero un qbit per l'overflow, tre per i test parziali e uno per il test globale ($\text{DIM} = \text{StateDIM} + \text{QueenDIM} + \text{ChecksDIM} + \text{AuxDIM} + 5$);
- **get_dims**: chiamata di tutti i passi precedenti per un dato N .

Viene quindi istanziato un registro di dimensione DIM, con la seguente sequenza ordinata di qbits:

```
Register = {State_Qbits,
            XOR_Qbits,
            Overflow_Qbit,
            Check_Qbits,
            Partial_Test_Qbits,
            Test_Qbit,
            Aux_Qbits}
```

3.2. Queens split

La lista dei qbits di stato viene trasformata, mediante la clausola **queens_split**, in una lista di sotto-liste, in cui ogni sotto-lista rappresenta lo stato di una singola regina; questa trasformazione serve a semplificare la gestione dei confronti tra le regine.

Si passa quindi dalla seguente rappresentazione:

State_Qbits = [q(0), q(1), ..., q(StateDIM-1)],

alla seguente:

Queens = [Queen(0), Queen(1), ..., Queen(N-1)],

dove il generico elemento Queen(k), con $k=[0, N-1]$, è la lista di qbits di dimensione QueenDIM relativa alla k-esima regina.

3.3. Parametrizzazione

Il registro, la lista Queens e altri parametri (qbits di controllo, qbits target, qbits ausiliari e sotto-circuiti modulari) vengono restituiti dalla clausola **init**, per poi essere passati alle opportune clausole dei componenti della logica circuitale; tali clausole, sono approfondite nei successivi paragrafi.

4. Vincolo sulle colonne

Il vincolo sulle colonne viene realizzato con le seguenti clausole:

- **col_eq**: costruzione del circuito parziale relativo ai confronti di uguaglianza sulle colonne per una data regina con tutte le successive;
- **eq_conj**: completamento del circuito compute-test-uncompute di uno dei vincoli, dato il relativo circuito di compute;
- **col_eq_conj**: completamento del circuito di compute chiamando **col_eq** per ogni regina, eccetto l'ultima, e applicazione dell'operatore di coniugazione chiamando **eq_conj**.

In particolare, per l'operatore di uguaglianza si è costruito un sotto-circuito modulare, partendo dall'osservazione che la struttura del sotto-operatore NOR fosse riutilizzabile, ad eccezione dell'azione sul target del confronto; tale modulo viene realizzato dalla clausola **nor_subcircuit** che restituisce soltanto la fase di compute del NOR. La clausola **init** si occupa di passare questo modulo e il suo inverso come argomenti di input a **col_eq_conj**; in **col_eq** vengono quindi congiunti all'azione sul target, diversa per ogni confronto.

4.1. Caso 4x4

In figura viene riportato, per il caso 4x4, il sotto-circuito Prolog e il corrispondente sotto-circuito Qiskit relativi alla chiamata di **col_eq** per la regina **Q0**.

La mappatura tra i qbits di Prolog e Qiskit è la seguente:

Stato: $q(0) - q(7) \leftrightarrow Q0 - Q3$

Ancilla: $q(8) - q(20) \leftrightarrow A0 - A12$

La sequenza di operazioni è la seguente:

1. fanout di Q0 sugli XOR_Qbits (q(8), q(9));
2. XOR bitwise di Q0 con Q1, con memorizzazione in-place negli XOR_Qbits;
3. NOR (NOT seguito da AND) degli XOR_Qbits con target action nel primo qbit non utilizzato dei Check_Qbits (q(11) - q(16)).
4. uncompute dei NOT e dello XOR bitwise;
5. ripetere le operazioni analoghe dal passo 2 per le regine successive (Q2, Q3);
6. uncompute del fanout di Q0.

I passi 2 e 3 compongono l'operatore di uguaglianza (EQ).

```
[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],          % Q0 FANOUT

[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(11):[[tof(q(8),q(9),q(11))]]],                             % Q0 EQ Q1
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],          %

[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(12):[[tof(q(8),q(9),q(12))]]],                             % Q0 EQ Q2
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],          %

[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(13):[[tof(q(8),q(9),q(13))]]],                             % Q0 EQ Q3
[q(8):[[not(q(8))]],q(9):[[not(q(9))]]],                      %
[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],          %

[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],          % Q0 FANOUT
```

Figura 1 - Caso 4x4: vincolo sulle colonne per Q0 (Prolog)

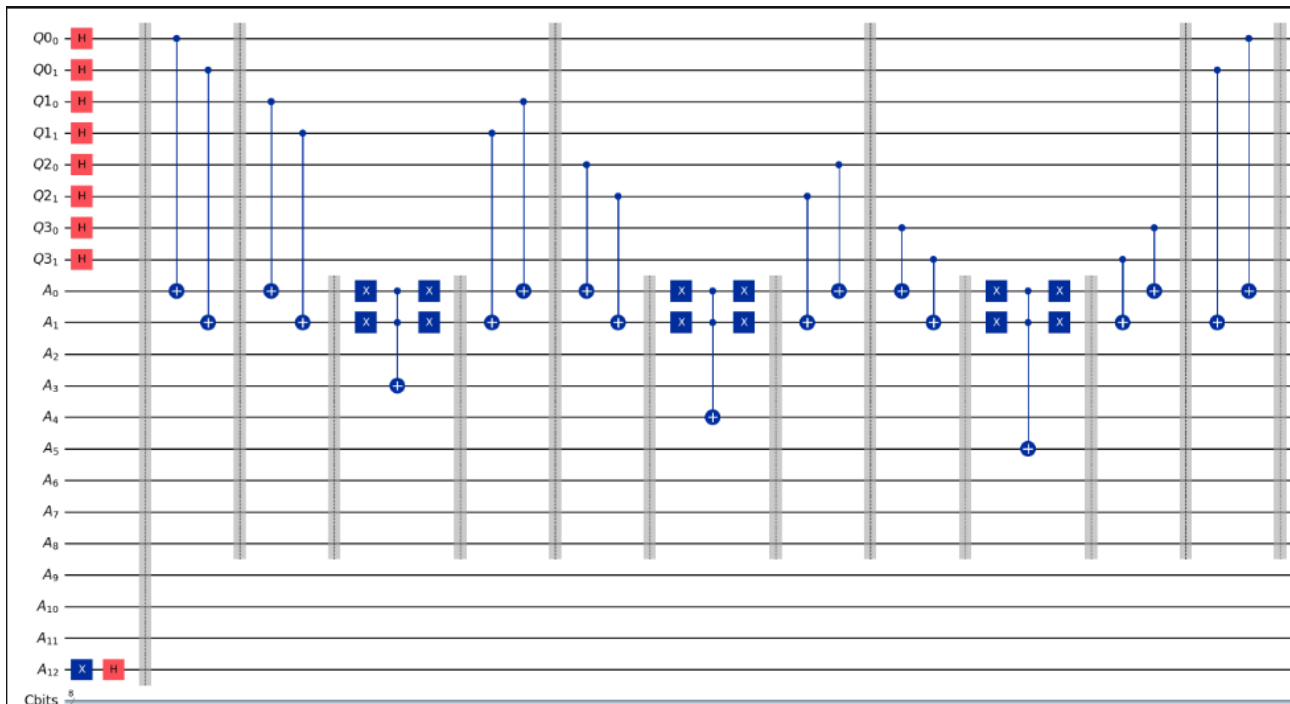


Figura 2 - Caso 4x4: vincolo sulle colonne per Q0 (Qiskit)

Il sotto-circuito viene ripetuto successivamente per Q1 e Q2 in modo analogo per completare la fase di compute relativa al vincolo sulle colonne.

Successivamente viene effettuato il test di soddisfacimento del vincolo sulle colonne e l'uncompute del componente (rendendo in tal modo riutilizzabili i Check_Qbits).

5. Vincolo sulle diagonali

Il vincolo sulle diagonali viene realizzato con le seguenti clausole:

- **diag_eq**: costruzione del circuito parziale relativo ai confronti di uguaglianza sulle diagonali per una data regina, con accumulazione di un offset di colonna, con tutte le successive, e "reset" dell'offset accumulato al termine dei confronti;
- **eq_conj**: completamento del circuito compute-test-uncompute di uno dei vincoli, dato il relativo circuito di compute;
- **diag_eq_conj**: completamento del circuito di compute chiamando **diag_eq** per ogni regina, eccetto l'ultima, e applicazione dell'operatore di coniugazione chiamando **eq_conj**.

Il vincolo sulle diagonali viene costruito due volte, una per le diagonali differenza (offset di colonna positivo) e una per le diagonali somma (offset di colonna negativo);

In particolare, per l'offset di colonna si è costruito un sotto-circuito modulare, partendo dall'osservazione che la struttura di incremento/decremento fosse riutilizzabile; tale modulo viene realizzato dalla clausola **incr_subcircuit** che restituisce il circuito di incremento (con controllo di overflow).

La clausola **init** si occupa di restituire questo modulo e il suo inverso, che sono poi passati nelle due

chiamate di **diag_eq_conj**, in cui si scambiano essenzialmente di ruolo come parametri (l'inverso dell'incremento è il decremento).

Infine, analogamente a quanto visto per il vincolo sulle colonne, anche in questo caso viene utilizzato il modulo **nor_subcircuit** per l'operatore di uguaglianza.

5.1. Caso 4x4

In figura viene riportato, per il caso 4x4, il sotto-circuito Prolog e il corrispondente sotto-circuito Qiskit relativi alla chiamata di **diag_eq** per la regina **Q0** nel caso del vincolo sulle diagonali differenza.

La mappatura tra i qbits di Prolog e Qiskit è la seguente:

$q(0) - q(7) \leftrightarrow Q0_0 - Q3_1$

$q(8) - q(20) \leftrightarrow A0 - A12$

$q(21) \rightarrow \emptyset$ (Qiskit gestisce internamente le ancille necessarie alle catene Toffoli)

La sequenza di operazioni è la seguente:

1. fanout di Q0 sugli XOR_Qbits (q(8), q(9));
2. incremento del valore di colonna di Q0 (offset +1) sugli XOR_Qbits, con qbit di controllo overflow (q(10)).
3. XOR bitwise di (Q0 + 1) con Q1, con memorizzazione in-place negli XOR_Qbits;
4. NOR (NOT seguito da AND) degli XOR_Qbits con target action nel primo qbit non utilizzato dei Check_Qbits (q(11) – q(16)).
5. uncompute dell'ancilla ausiliaria q(21), dei NOT e dello XOR bitwise;
6. ripetere le operazioni analoghe dal passo 2 per le regine successive (Q2, Q3);
7. uncompute dell'offset accumulato
8. uncompute del fanout di Q0.

Il passo 2 utilizza il modulo di incremento, il passo 7 il modulo di decremento (l'inverso).

I passi 2 e 3 compongono l'operatore di uguaglianza (EQ).

```

[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],          % Q0 FANOUT

[q(10):[[tof(q(8),q(9),q(10))]]],                               %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0 + 1
[q(8):[[not(q(8))]]],                                           %

[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(11):[[tof(q(10),q(21),q(11))]]],                             % (Q0 + 1) EQ Q1
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],          %

[q(10):[[tof(q(8),q(9),q(10))]]],                               %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0 + 2
[q(8):[[not(q(8))]]],                                           %

[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(12):[[tof(q(10),q(21),q(12))]]],                             % (Q0 + 2) EQ Q2
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],          %

```

Figura 3 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 1 (Prolog)

```

[q(10):[[tof(q(8),q(9),q(10))]]],                               %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0 + 3
[q(8):[[not(q(8))]]],                                           %

[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],          %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(13):[[tof(q(10),q(21),q(13))]]],                             % (Q0 + 3) EQ Q3
[q(21):[[tof(q(8),q(9),q(21))]]],                               %
[q(8):[[not(q(8))]],q(9):[[not(q(9))]],q(10):[[not(q(10))]]], %
[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],          %

[q(8):[[not(q(8))]]],                                           %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0 + 2
[q(10):[[tof(q(8),q(9),q(10))]]],                               %

[q(8):[[not(q(8))]]],                                           %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0 + 1
[q(10):[[tof(q(8),q(9),q(10))]]],                               %

[q(8):[[not(q(8))]]],                                           %
[q(9):[[cnot(q(8),q(9))]]],                                     % Q0
[q(10):[[tof(q(8),q(9),q(10))]]],                               %

[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],          % Q0 FANOUT

```

Figura 4 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 2 (Prolog)

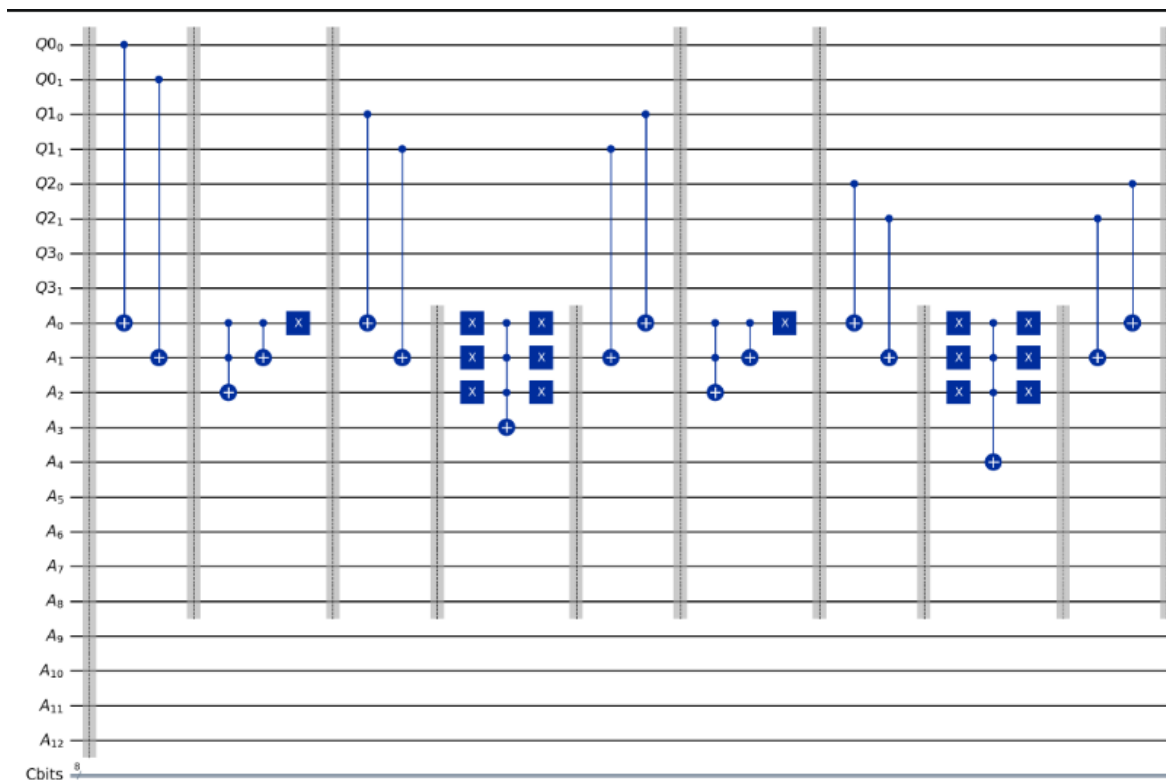


Figura 5 - Caso 4x4: vincolo sulle diagonali differenza per $Q0$, parte 1 (Qiskit)

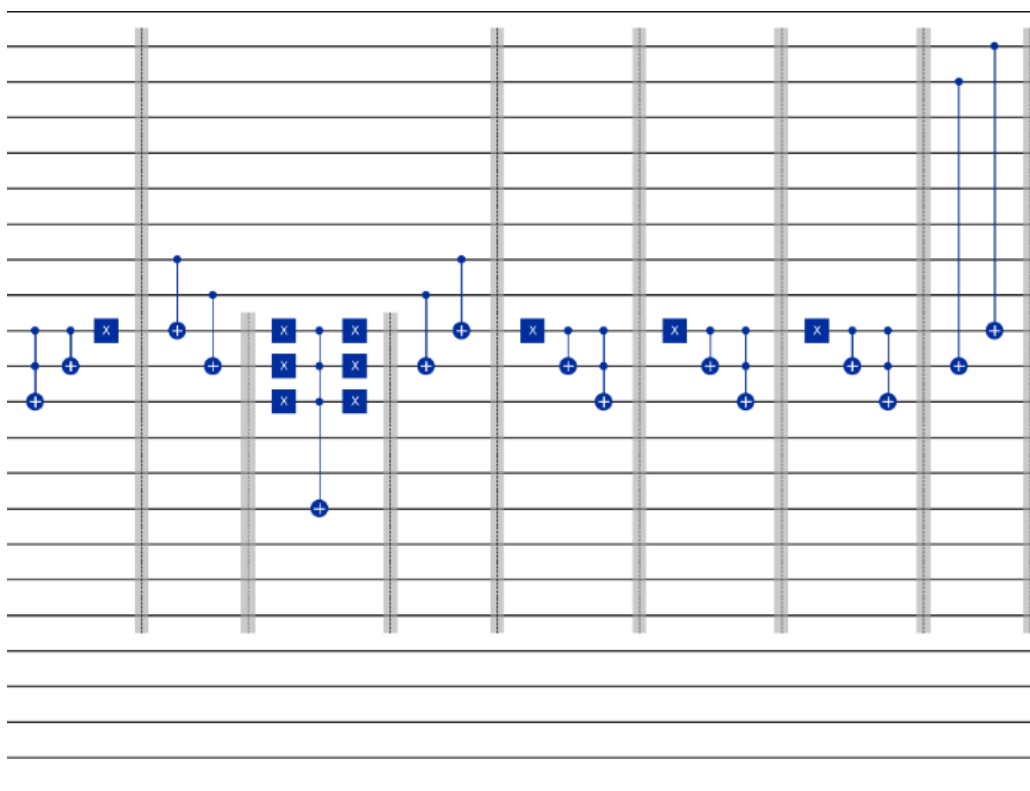


Figura 6 - Caso 4x4: vincolo sulle diagonali differenza per $Q0$, parte 2 (Qiskit)

Il sotto-circuito viene ripetuto successivamente per Q1 e Q2 in modo analogo per completare la fase di compute relativa al vincolo sulle diagonali differenza.

Successivamente viene effettuato il test di soddisfacimento del vincolo sulle diagonali differenza e l'uncompute del componente (rendendo in tal modo riutilizzabili i Check_Qbits).

Per il vincolo sulle diagonali somma si procede in modo analogo, ma con il ruolo invertito dei moduli di incremento e decremento.

6. Test di soddisfacimento dei vincoli

Al termine della fase di compute e prima della fase di uncompute di ciascuno dei tre vincoli, viene effettuato un test di soddisfacimento di vincolo con l'implementazione di un NOR, che verifica che nessuna collisione (o uguaglianza) tra regine sia avvenuta rispetto allo specifico vincolo.

L'operatore NOR presenta $N*(N-1)/2$ controlli (i Check_Qbits) e memorizza il risultato in uno dei tre qbits per i test parziali (Partial_Test_Qbits).

Al termine della fase di compute globale e prima della fase di uncompute globale, viene effettuato un test globale di soddisfacimento dei vincoli con l'implementazione di un AND, che presenta tre controlli (i Partial_Test_Qbits) e memorizza il risultato nel qbit per il test globale (Test_Qbit).

6.1. Caso 4x4

In figura vengono riportati, per il caso 4x4, i sotto-circuiti Prolog e i corrispondenti sotto-circuiti Qiskit relativi in ordine al primo test parziale (gli altri due sono analoghi) e al test globale.

La mappatura tra i qbits di Prolog e Qiskit è la seguente:

$q(0) - q(7) \leftrightarrow Q0_0 - Q3_1$

$q(8) - q(20) \leftrightarrow A0 - A12$

$q(21) - q(24) \rightarrow \emptyset$ (Qiskit gestisce internamente le ancille necessarie alle catene Toffoli)

La sequenza di operazioni per il primo test parziale è la seguente:

1. Operatore NOR con controlli i Check_Qbits ($q(11) - q(16)$) e target $q(17)$, il primo dei Partial_Test_Qbits ($q(17) - q(19)$);
2. Uncompute degli Aux_Qbits ($q(21) - q(24)$) e dei NOT.

La sequenza di operazioni per il test globale è la seguente:

1. Operatore AND con controlli i Partial_Test_Qbits e target il Test_Qbit ($q(20)$);
2. Uncompute dell'ancilla ausiliaria $q(21)$.

```
[q(11):[[not(q(11))]],q(12):[[not(q(12))]],q(13):[[not(q(13))]],q(14):[[not(q(14))]],q(15):[[not(q(15))]],q(16):[[not(q(16))]]],
[q(21):[[tof(q(11),q(12),q(21))]]],
[q(22):[[tof(q(13),q(21),q(22))]]],
[q(23):[[tof(q(14),q(22),q(23))]]],
[q(24):[[tof(q(15),q(23),q(24))]]],
[q(17):[[tof(q(16),q(24),q(17))]]],
[q(24):[[tof(q(15),q(23),q(24))]]],
[q(23):[[tof(q(14),q(22),q(23))]]],
[q(22):[[tof(q(13),q(21),q(22))]]],
[q(21):[[tof(q(11),q(12),q(21))]]],
[q(11):[[not(q(11))]],q(12):[[not(q(12))]],q(13):[[not(q(13))]],q(14):[[not(q(14))]],q(15):[[not(q(15))]],q(16):[[not(q(16))]]],
```

Figura 7 - Caso 4x4: test parziale (Prolog)

```
[q(21):[[tof(q(17),q(18),q(21))]]],
[q(20):[[tof(q(19),q(21),q(20))]]],
[q(21):[[tof(q(17),q(18),q(21))]]]
```

Figura 8 - Caso 4x4: test globale (Prolog)

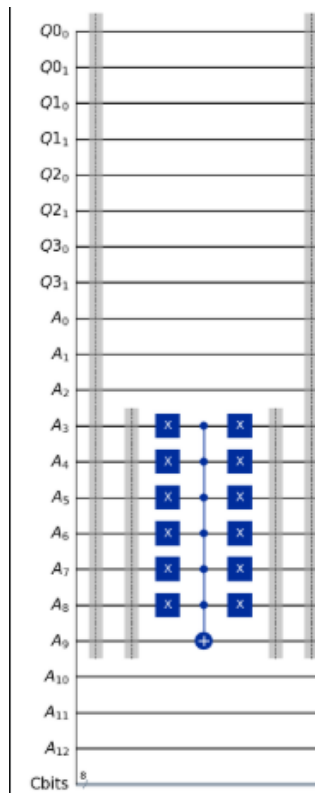


Figura 9 - Caso 4x4:
test parziale (Qiskit)

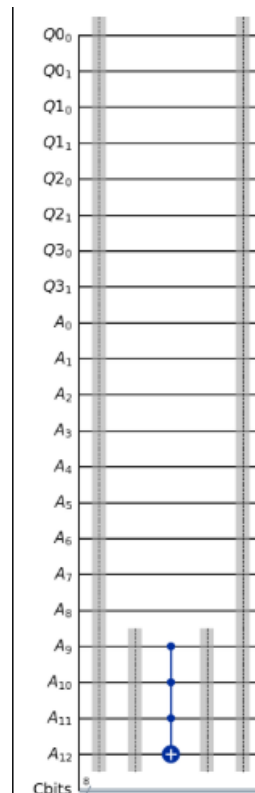


Figura 10 - Caso 4x4:
test globale (Qiskit)

7. Composizione dell'oracolo

Il circuito finale, che rappresenta il queens problem oracle, viene restituito, per un dato N, dalla clausola **queens_problem**, che svolge in ordine le seguenti chiamate:

- **init**: per un dato N, restituisce tutti i parametri necessari ai componenti circuitali;
- **col_eq_conj**: restituisce il circuito compute-test-uncompute per il vincolo sulle colonne;
- **diag_eq_conj**: restituisce il circuito compute-test-uncompute per il vincolo sulle diagonali differenza;
- come il passo precedente, ma per il vincolo sulle diagonali somma;
- **test**: restituisce il circuito per il test globale;

Infine, esegue la concatenazione dei componenti e dell'uncompute globale.

La clausola **queens_problem** è anche l'interfaccia di utilizzo per effettuare query prolog su N arbitrari.

Se **queens_problem** viene interrogato nella forma “?- queens_problem(N,Circuit).”, restituisce il circuito dell'oracolo.

Se **queens_problem** viene interrogato nella forma

“?- queens_problem(N,Circuit1,Circuit2,Circuit3,Circuit4).”, restituisce in ordine i componenti circuitali (i tre vincoli e il test globale).

7.1. Caso 4x4

In figura viene riportato, per il caso 4x4, la composizione dell'oracolo in Qiskit.

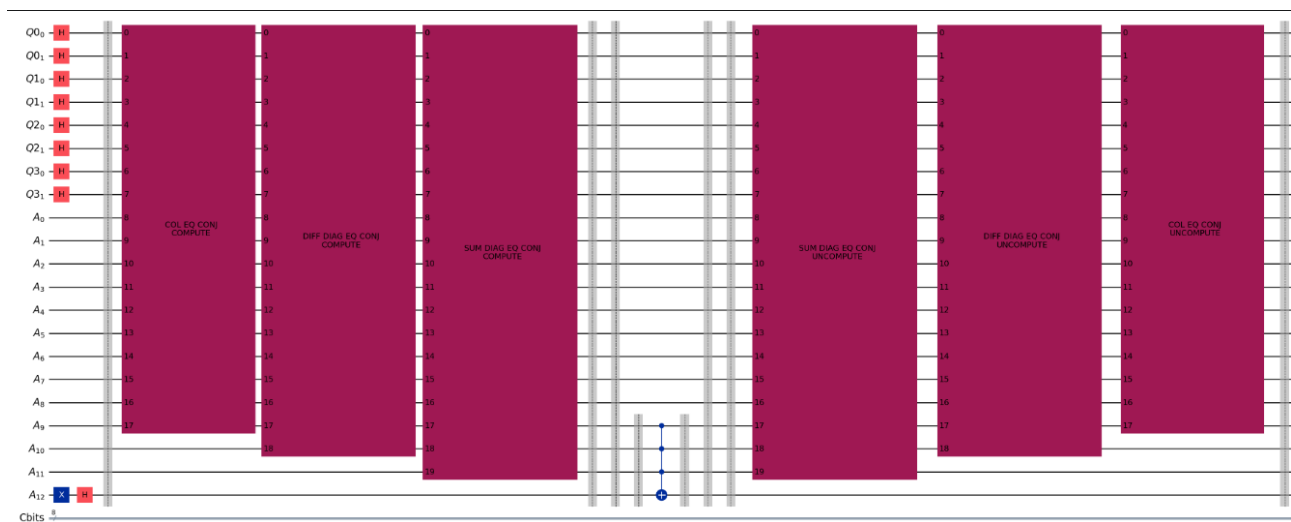


Figura 11 - Caso 4x4: composizione dell'oracolo (Qiskit)